UNE QUESTION AVANT DE COMMENCER

Où se cache ce que sait une IA ?

Un modèle de langage produit du texte. Imaginons qu'il explique une notice sur un petit ordinateur, sans connexion.

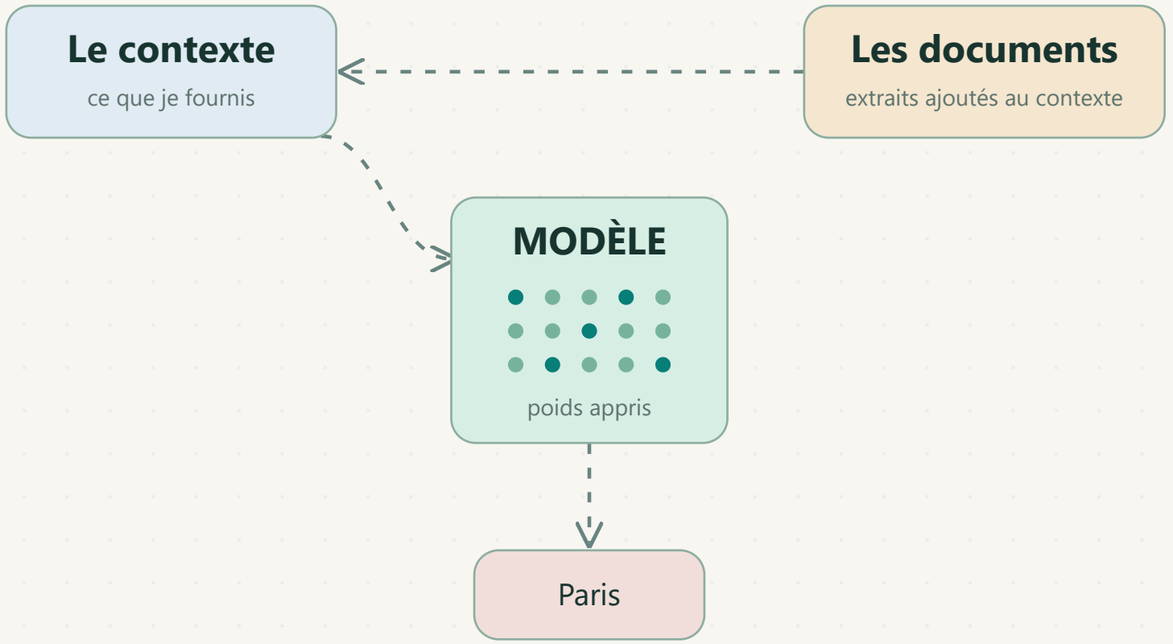
> **Poids :** réglages numériques appris.

> **Contexte :** texte reçu pour cette réponse.

> **Documents :** extraits qu'un outil peut ajouter au contexte.

À RETENIR

Les informations de la réponse peuvent venir de plusieurs endroits.



Les extraits consultés rejoignent le contexte.

Parcours découverte : aucun prérequis en IA. Les codes R ouvrent les références ; le bouton Ingénieur donne les détails.

DEUX MOMENTS À NE PAS CONFONDRE

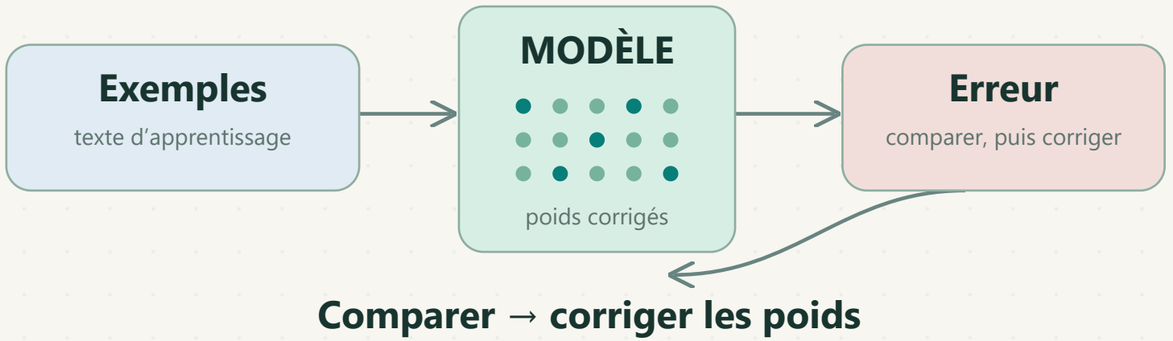
Apprendre. Puis utiliser.

Apprendre, c'est modifier les réglages à partir d'exemples. Utiliser le modèle, c'est calculer une réponse avec ces réglages.

> **Pré-entraînement :** premier apprentissage sur un grand ensemble de données.

> **Ajustement fin :** entraînement supplémentaire pour adapter le modèle.

> Les poids, biais et autres réglages appris sont des paramètres ; ils ne changent pas dans l'inférence ordinaire.



ENSUITE : UNE INFÉRENCE

nouveau contexte → poids fixes → sortie

À RETENIR

Les poids sont des paramètres. D'autres réglages appris, comme les biais, en font aussi partie.

Un biais est une valeur ajoutée dans un calcul. Ajouter du contexte ne signifie pas réentraîner le modèle.

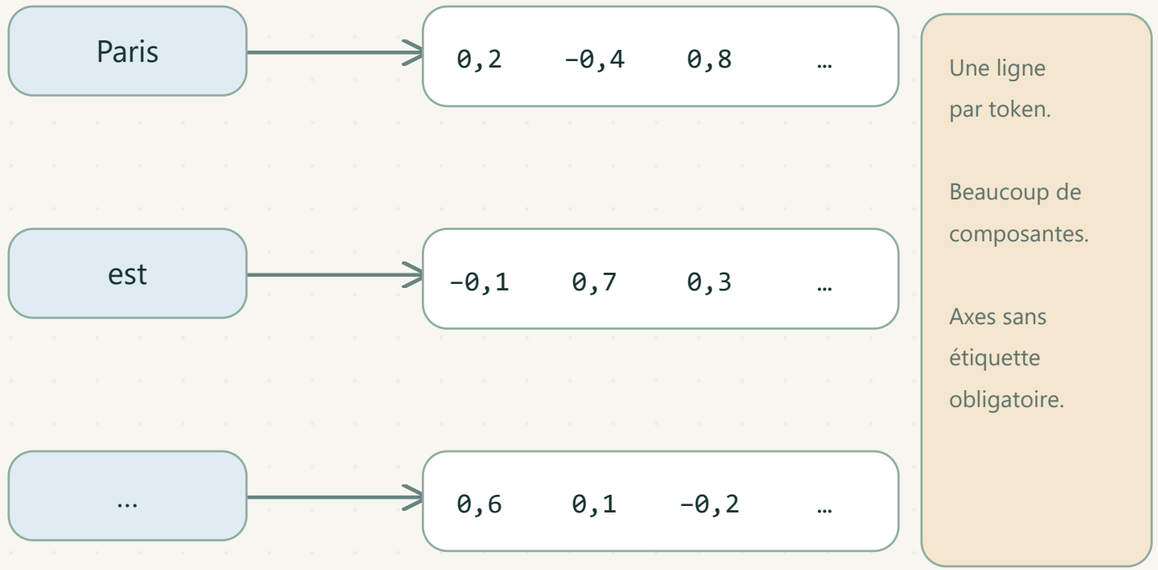
DU TEXTE AUX NOMBRES

Pour la machine, le texte devient des nombres.

Le programme découpe le texte en morceaux appelés tokens, puis associe des nombres à chaque morceau.

> Un token peut être un mot, un bout de mot ou un signe.

> La liste de nombres associée au token s'appelle un **vecteur** ; sa représentation apprise, un **embedding**.



Découpage et nombres illustratifs. Ils ne proviennent pas d'un modèle réel.

À RETENIR

Le modèle travaille sur ces nombres, pas sur des mots rangés comme dans un dictionnaire.

LE CONTEXTE CHANGE L'INTERPRÉTATION

« Avril » : qui ou quand ?

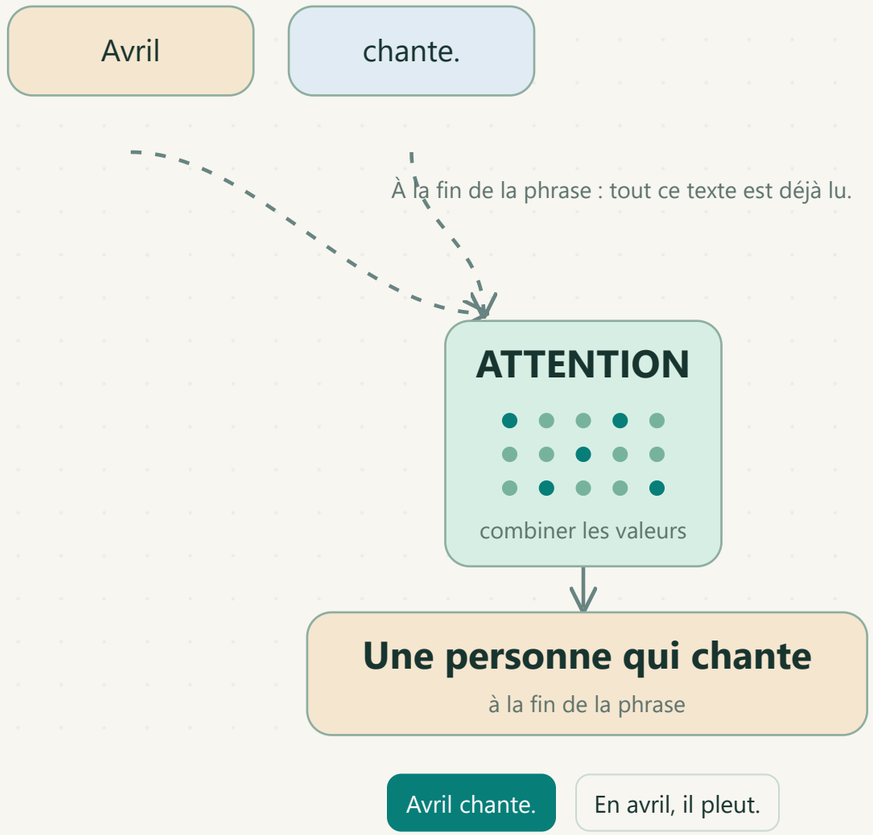
L'attention aide à combiner les informations déjà lues qui comptent pour la position traitée.

> À la fin de « Avril chante », la phrase suggère une personne.

> Dans « En avril, il pleut », elle suggère un mois.

À RETENIR

À une position donnée, le modèle causal étudié ne consulte pas les tokens futurs.



Exemple d'interprétation en fin de phrase. Les liens sont dessinés, pas mesurés. La formule Q/K/V est dans le parcours Ingénieur.

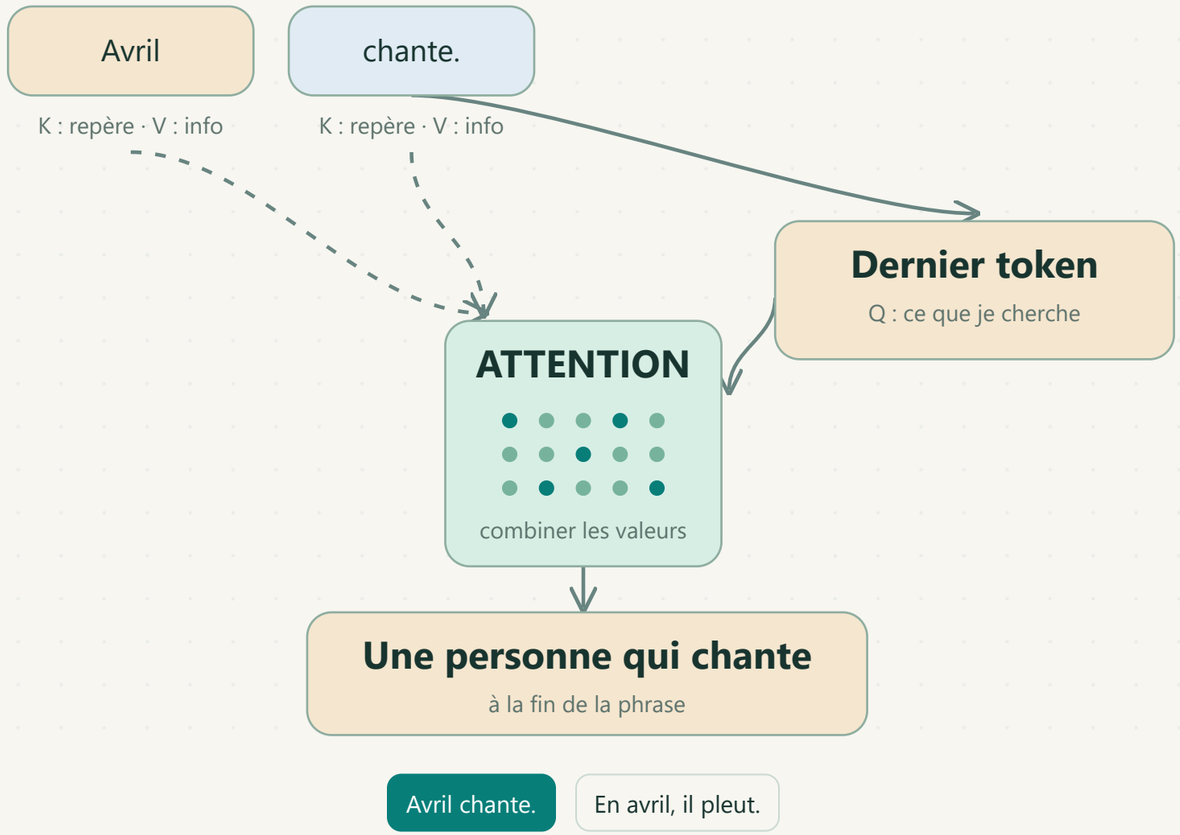
APPROFONDISSEMENT INGÉNIEUR

Comment calculer l'attention ?

Un Transformer enchaîne des blocs d'attention, qui combinent le contexte, et des transformations de vecteurs.

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$$

Q : requêtes, ce que l'on cherche ; K : clés à comparer.
V : valeurs, informations à combiner ; d_k : taille d'une clé.
^{T} échange lignes et colonnes. softmax donne des pondérations positives dont la somme vaut 1.



Interprétation illustrée à la fin de la phrase, pas au token « Avril » isolé. Liens dessinés, pas mesurés dans un modèle.

À RETENIR

Un modèle causal ne lit pas de token futur. À la fin de la phrase, toute la phrase est déjà disponible.

UNE REPRÉSENTATION PARTAGÉE

Un neurone n’est pas un tiroir à faits.

Un neurone combine des entrées. Les poids se rangent en matrices, des tableaux de nombres.

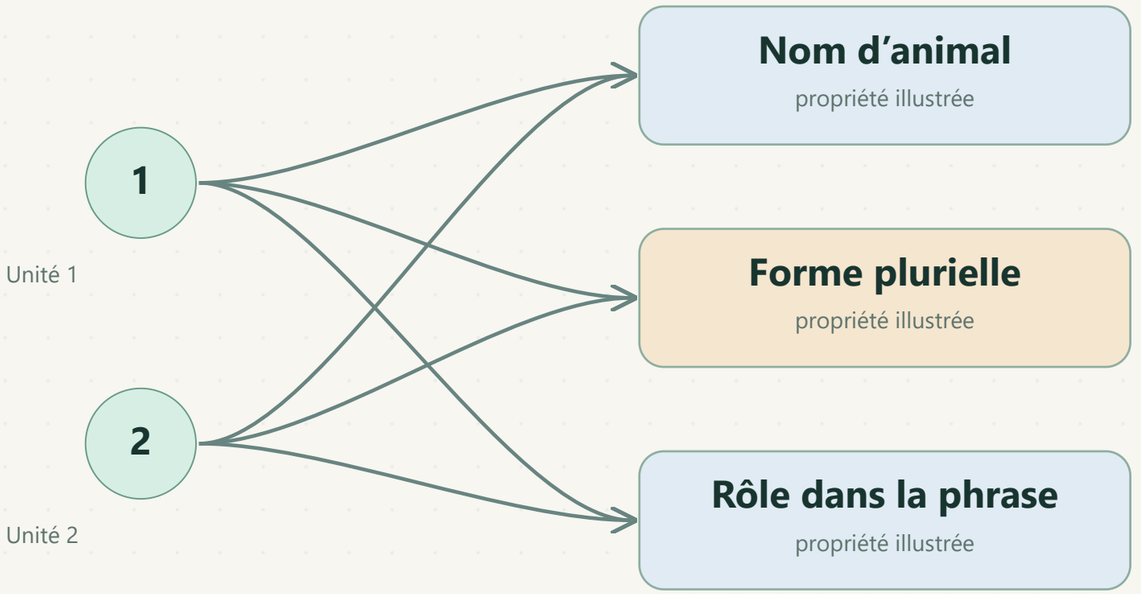
> **MLP :** perceptron multicouche, un réseau de couches de neurones.

> **Activation :** fonction qui transforme le résultat d’un neurone.

> **Superposition :** des propriétés partagent des dimensions de représentation.

À RETENIR

Une propriété peut mobiliser plusieurs unités. Ce partage n’empêche pas la mémorisation de passages.



Deux unités partagées par trois propriétés illustrées.

Deux unités et trois propriétés illustrées : ce partage peut créer des interférences. Ce n’est pas une carte réelle de neurones.

À VOUS DE PRÉDIRE

J’ajoute un nom dans mon message.

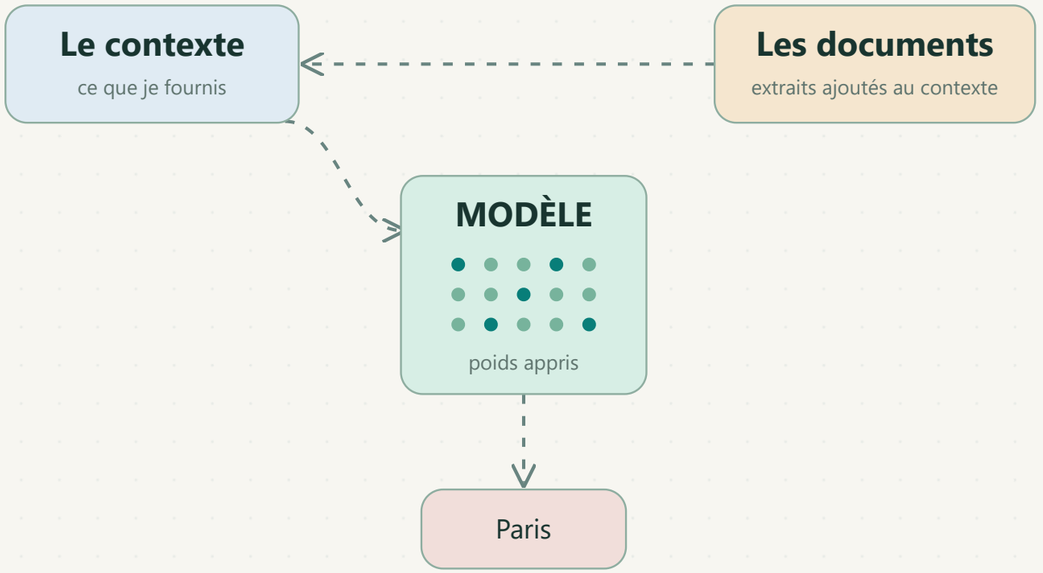
Le modèle réutilise ce nom. Quelle explication suffit sans supposer de réentraînement ?

A. Le contexte peut suffire à expliquer sa réponse.

B. Les poids ont été modifiés par ce message.

C. Le nom a été enregistré durablement pour tous.

La réponse peut utiliser le contexte. À elle seule, elle ne prouve ni mise à jour des poids ni mémoire durable.



Les extraits consultés rejoignent le contexte.

À RETENIR

Poids appris, contexte temporaire et stockage de l’application sont trois choses différentes.

Une réponse correcte ne suffit pas à identifier le mécanisme de mémorisation.

DEUX PHASES DE L'INFÉRENCE

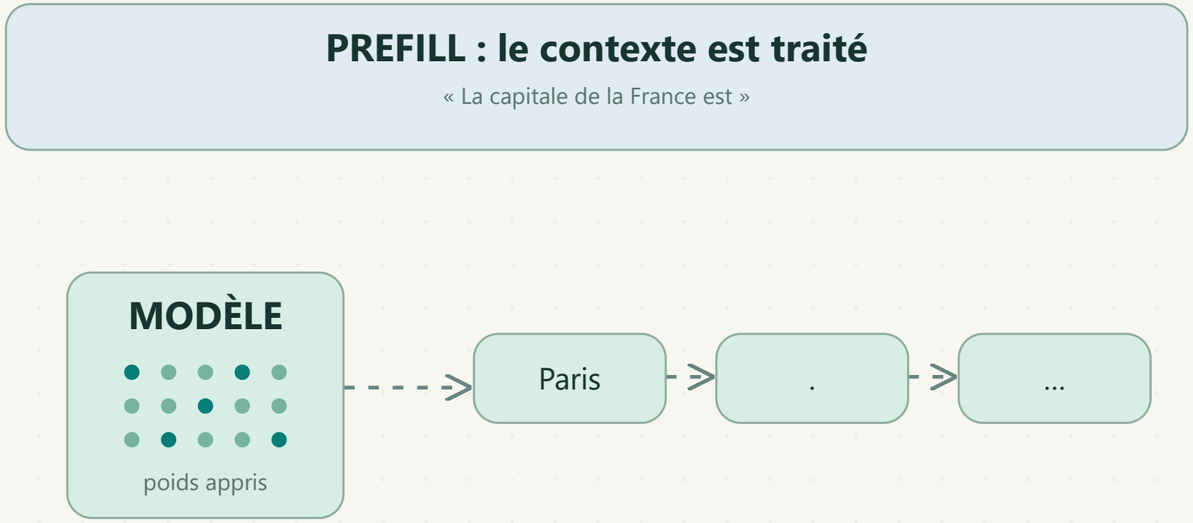
Lire le début. Puis continuer.

D'abord, le modèle traite le texte reçu. Ensuite, il produit la suite.

> **Prefill :** traiter de nombreuses positions avec davantage de réutilisation des poids.

> **Decode à petit lot :** moins de réutilisation ; les transferts peuvent dominer.

> Ce diagnostic n'est pas une loi : caches, taille du lot, architecture et cible peuvent changer le facteur limitant.



DECODE : une suite qui se construit.

À RETENIR

Lire l'entrée offre du travail parallèle. Produire la suite dépend souvent des transferts ; cela varie selon la machine et le modèle.

Le débit compte les tokens par seconde. Le coût réel dépend du modèle, du contexte, du nombre de requêtes et du matériel.

PREMIER BUDGET : LES POIDS

Sept milliards de nombres.

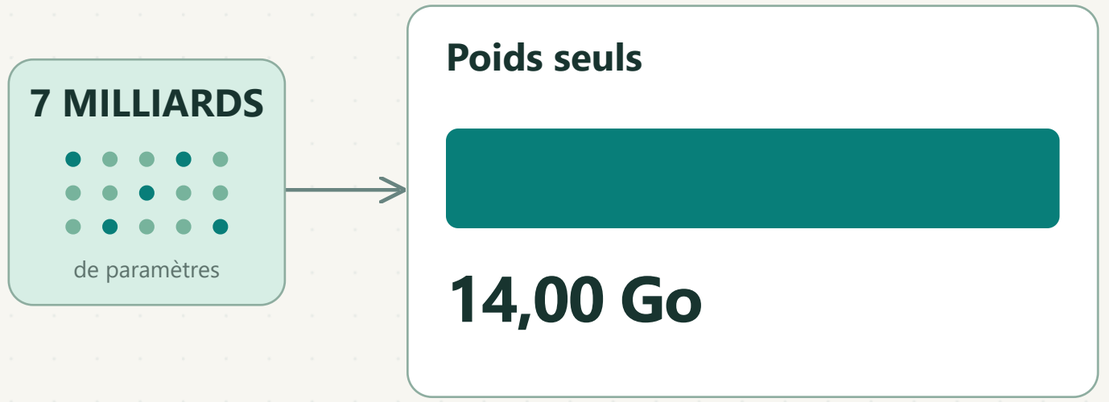
Un bit vaut 0 ou 1. Huit bits font un octet. Moins de bits par réglage réduit son volume de stockage.

> Exemple idéal : 7 milliards de paramètres.

> 16 bits par paramètre : **14 Go**. À 4 bits : **3,5 Go**.

À RETENIR

Cela compte les réglages seuls. Ce n'est pas encore la mémoire totale de l'application.



16 bits par paramètre · compte idéal

16 bits | 8 bits | 4 bits | 2 bits

Go = milliard d'octets. Échelles, format du fichier et autres données s'ajoutent. La formule est dans le parcours Ingénieur.

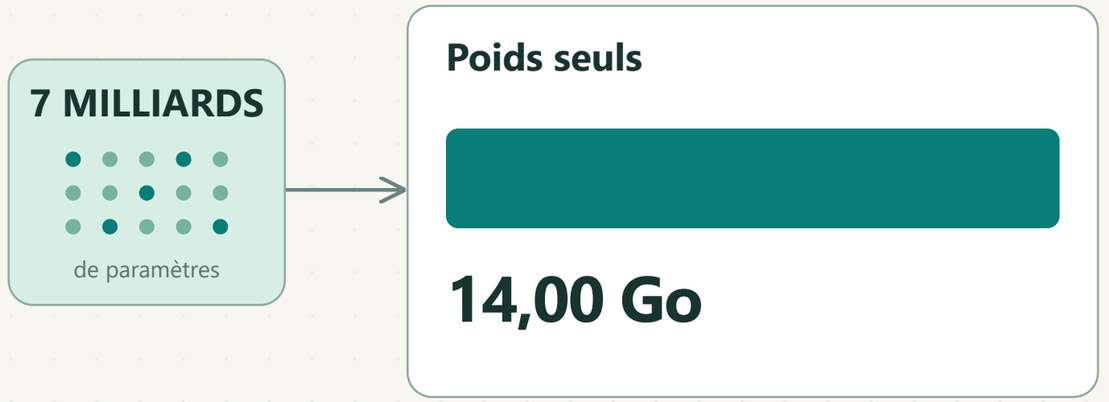
APPROFONDISSEMENT INGÉNIEUR

Combien d'octets pour les poids ?

Un bit vaut 0 ou 1. Huit bits font un octet. La précision choisie change le nombre de bits par poids.

$$M_{poids} = N \times b / 8$$

N : nombre de paramètres, les réglages appris.
b : bits par paramètre ; M : taille en octets.
Ici $N = 7 \times 10^9$; 1 Go $= 10^9$ octets.



16 bits par paramètre · compte idéal

16 bits | 8 bits | 4 bits | 2 bits

Poids seuls : ni la RAM totale (mémoire de travail), ni un fichier complet. La barre change avec les bits.

À RETENIR

Le programme, ses données de travail et le format du fichier occupent aussi de la mémoire.

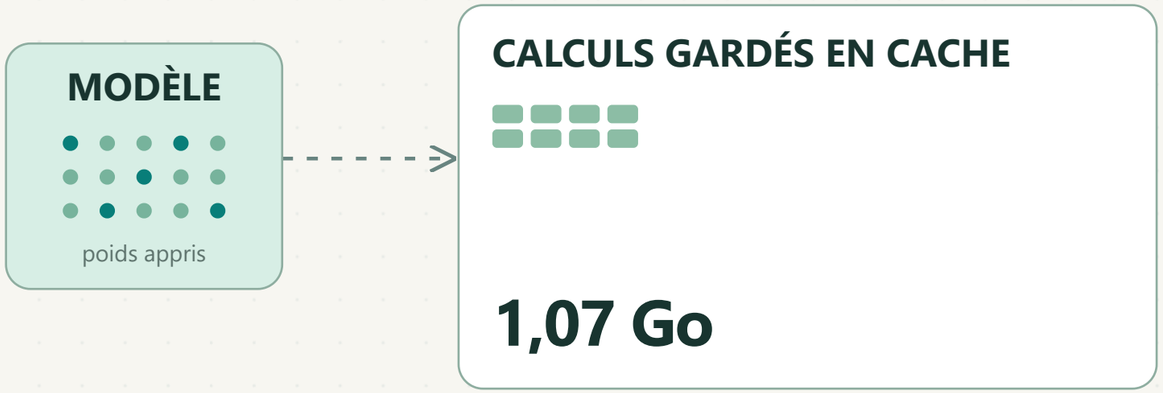
DEUXIÈME BUDGET : LE CONTEXTE

Le cache grandit avec l'histoire.

Pour continuer une phrase, le modèle réutilise certains calculs sur le texte déjà lu au lieu de tout refaire.

> Ce stockage de réutilisation s'appelle le **cache**.

> La **fenêtre de contexte** est la quantité de texte que le modèle peut prendre en compte.



8 192 tokens

Plus de contexte, plus de calculs à garder.

Tokens présents 8 192

Exemple calculé en Go, pas un essai sur Pi. K et V sont des représentations d'attention ; la formule et le partage des têtes sont dans les détails.

À RETENIR

Un contexte plus long peut demander plus de mémoire. Le cache grandit dans l'attention standard illustrée ici.

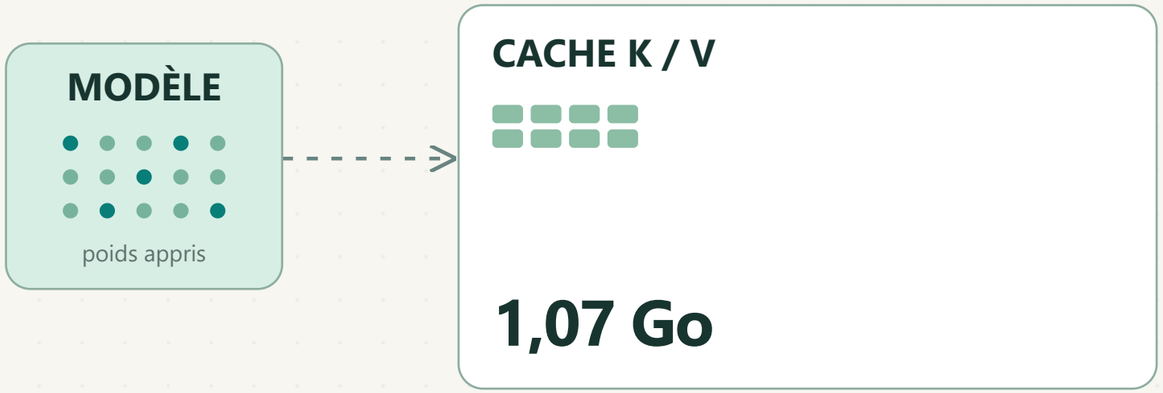
APPROFONDISSEMENT INGÉNIEUR

Compter le cache sans oublier les têtes.

Un cache est une mémoire de réutilisation. Celui de l'attention garde les clés K et valeurs V déjà calculées.

$$M_{KV} = 2 \times L \times S \times H_{KV} \times d \times p$$

Pour une séquence : L couches, S tokens,
H_{KV} têtes K/V, d composantes par tête,
p octets par composante. Le 2 compte K et V.



8 192 tokens · 8 têtes K/V

Plus de contexte : colonnes · plus de têtes : lignes

Tokens présents 8 192 8 têtes K/V · changer

Pour une séquence : L=32, d=128, p=2. 1 Go=10^9 octets ; 1 Gio=2^{30} octets. Pas une mesure sur Pi.

À RETENIR

Une tête est une branche d'attention. En GQA, plusieurs têtes de requêtes partagent les mêmes K/V.

LE BUDGET COMPLET

Le fichier tient. Et le reste ?

La RAM est la mémoire de travail. Elle doit aussi accueillir le contexte, les calculs en cours et les logiciels.

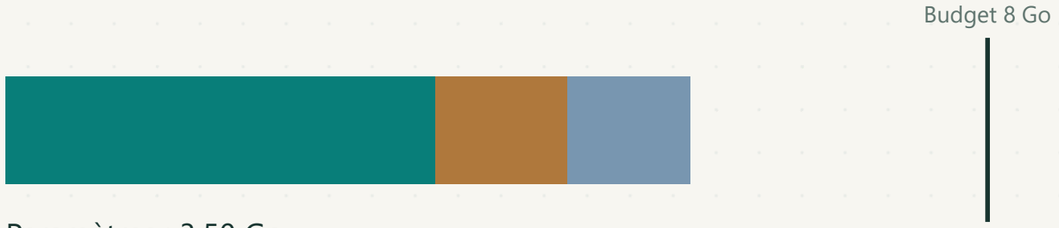
> Additionner **paramètres + cache + valeurs temporaires + logiciels et système**.

> Garder une marge ; vérifier ensuite le maximum réellement occupé.

À RETENIR

Le total doit rester sous le budget disponible. Une marge calculée doit encore être confirmée par une mesure.

RAM : un budget commun



Paramètres : 3,50 Go

Cache : 1,07 Go

Reste du système et du calcul : 1 Go supposé

TOTAL : 5,57 Go

Marge calculée : 2,43 Go

Budget : 8 Go · changer

Poids : 4 bits · changer

Calcul pédagogique : poids à 4 bits, cache selon le contexte ; 1 Go supposé pour le reste. Budgets de 4 ou 8 Go hypothétiques, pas une mesure de carte.

LE CALCUL ATTEND PARFOIS LES DONNÉES

Une usine rapide. Une route étroite.

Imaginons une machine : elle doit lire des données et effectuer des calculs. Le travail le plus lent peut dominer.

> La **latence** est la durée d'une étape.

> Réduire le temps de calcul aide peu si les données arrivent trop lentement.

À RETENIR

Réduire les octets effectivement transférés peut aider. Réduire seulement le fichier ne garantit pas ce gain.



Machine fictive, pas un Raspberry Pi : hypothèses réglables. Temps idéaux, cache et autres coûts exclus. La formule est dans les détails.

APPROFONDISSEMENT INGÉNIEUR

Une borne pour raisonner sur le coût.

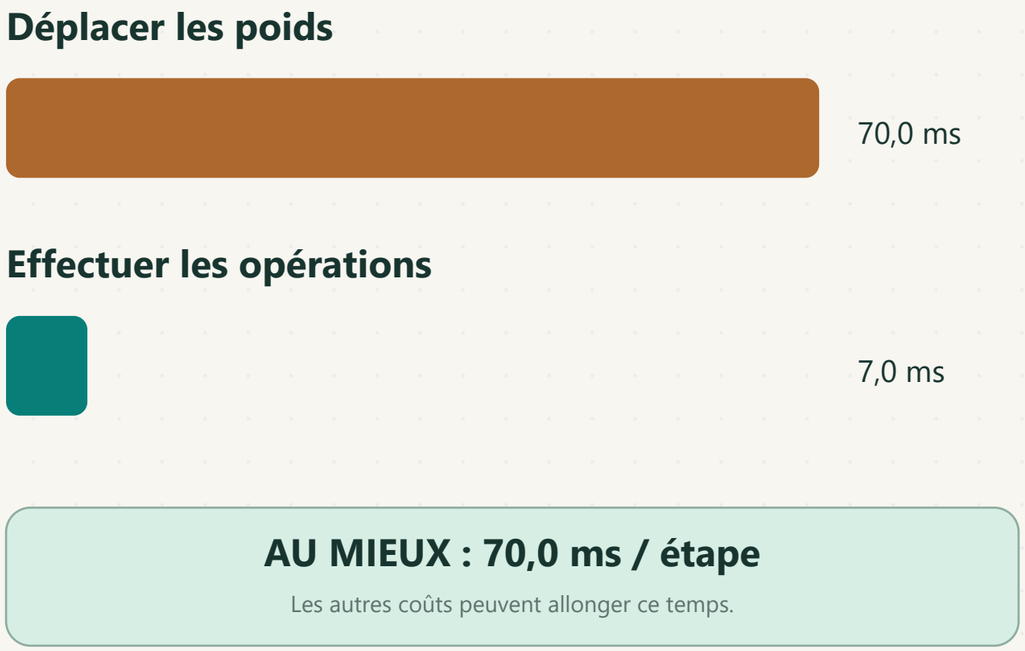
Le débit mémoire est la quantité de données transférable par seconde. Le calcul peut attendre ces données.

$$T \geq \max(D / B, F / C)$$

T : latence, durée d'une étape ; max retient le plus grand temps.
D : octets transférés ; B : débit mémoire en octets/s.
F : opérations ; C : capacité de calcul en opérations/s.

À RETENIR

Cette borne n'est pas une mesure. Changer la capacité de calcul peut déplacer le facteur limitant.



Débit mémoire — 50 Go/s | Poids 4 bits · changer | Calcul : rapide · changer

Hypothèses réglables, pas de chiffres matériels de Pi. $N=7\times10^9$; D=poids effectivement lus une fois ; F≈2N. Cache et surcoûts exclus.

UN DIAGNOSTIC AVANT UNE OPTIMISATION

Le calcul prend 7 ms. La lecture, 70 ms.

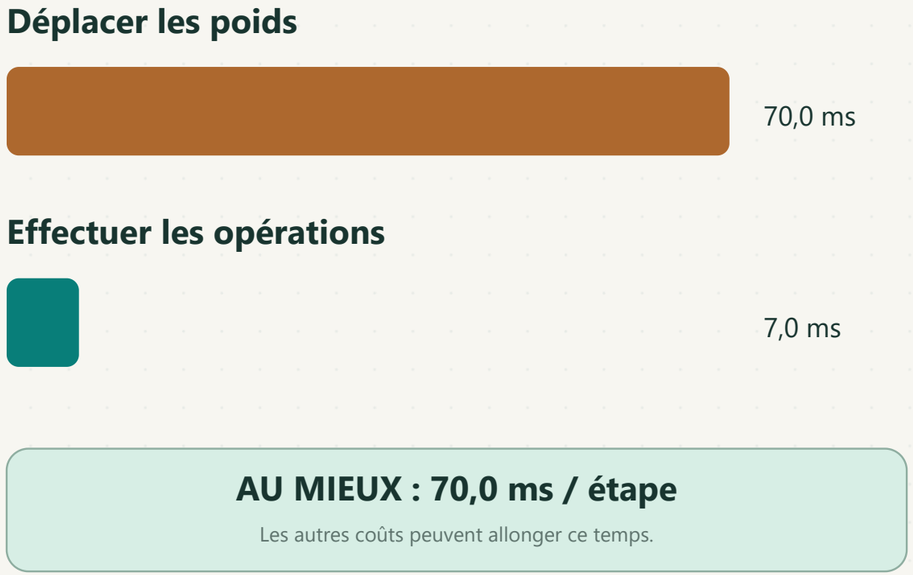
Dans le modèle idéal, on double seulement la capacité de calcul. Que devient la borne de temps ?

A. Elle passe à 35 ms.

B. Elle reste à 70 ms : les transferts dominent.

C. Elle passe à 7 ms.

La borne max(70,7) reste 70 ms quand 7 devient 3,5 ms. Une machine réelle ajoute d'autres coûts.



Nombres d'un exemple analytique, pas d'une mesure de PC ou de Raspberry Pi.

À RETENIR

Optimiser une brique peu coûteuse peut presque ne rien changer au temps total.

PISTE 1 · REPRÉSENTER AUTREMENT

Arrondir les poids. Observer la sortie.

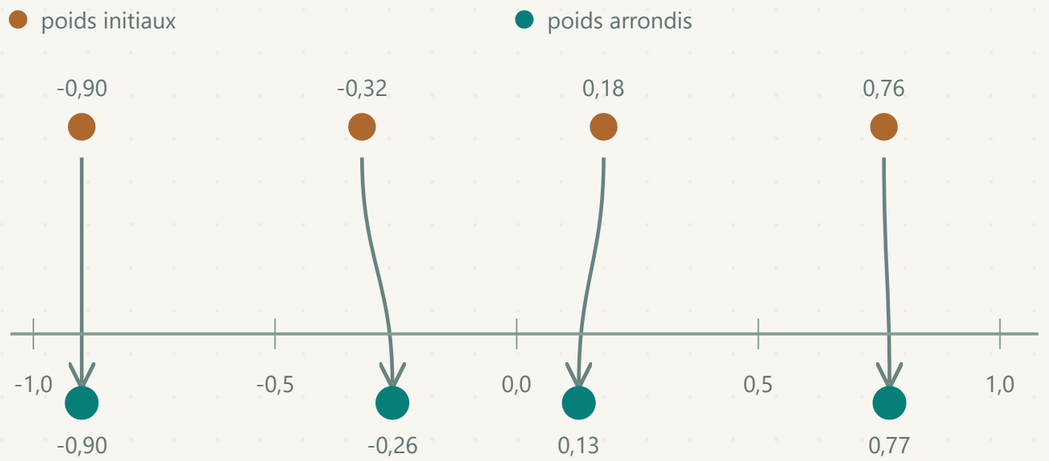
Quantifier, c’est remplacer des valeurs par quelques niveaux. Les arrondis peuvent changer la sortie.

> **Après entraînement :** convertir un modèle existant, puis vérifier les erreurs.

> **Entraînement adapté :** apprendre en tenant compte de la basse précision, comme dans BitNet.

À RETENIR

Cette animation montre l’écart dû aux arrondis. Elle ne mesure pas un gain de vitesse.



Valeur avant : -1,0500
avant arrondi des poids

Après : -0,9643
écart dû aux arrondis : 0,0857

8 bits | 4 bits | 2 bits

Exemple calculé dans la page. Les entrées restent les mêmes ; seul l’arrondi des poids change. Ce n’est pas l’exécution d’un modèle de langage.

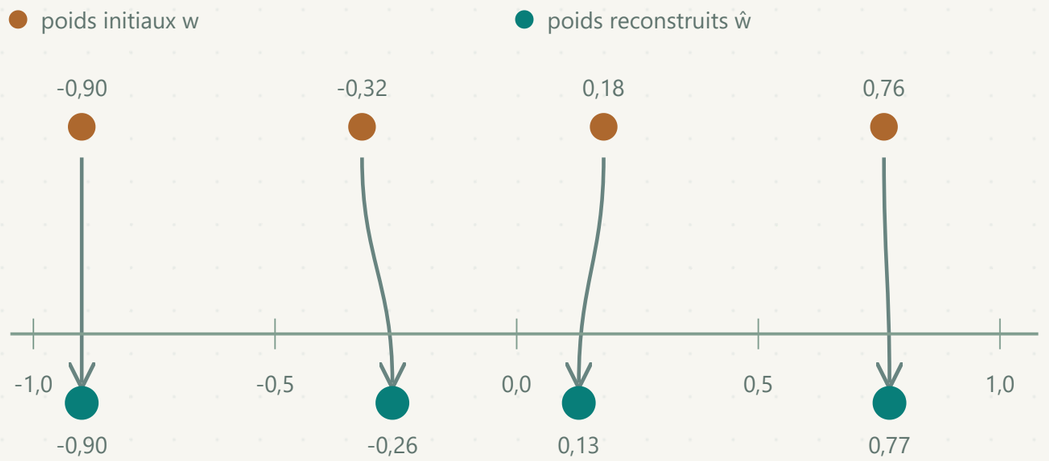
APPROFONDISSEMENT INGÉNIEUR

Quantifier. Calculer l'erreur.

Quantifier : remplacer des valeurs par quelques niveaux.

$$s = \max|w| \,/\, q_{max}$$
$$q = \text{borne}(\text{arrondi}(w/s), -q_{max}, q_{max})$$
$$\hat{w} = s \times q$$

b : bits ; $q_{max}=2^{b-1}-1$: plus grand code positif.
s : échelle, pas entre niveaux ; max|w| : plus grand poids sans signe.
w : poids ; q : code entier ; ŵ : poids reconstruit.
x : entrée ; y : sortie ; i : indice. Si max|w|=0 : s=1.



Valeur avant : -1,0500
y = somme des $w_i x_i$

Après : -0,9643
écart dû aux arrondis : 0,0857

8 bits | 4 bits | 2 bits

w=(−0,9 ; −0,32 ; 0,18 ; 0,76), x=(1 ; 0,5 ; −1 ; 0,25). Borner ramène au code autorisé ; arrondi à mi-distance loin de zéro.

À RETENIR

Calcul en flottants (réels approchés) : le stockage réduit ne promet pas d'accélération.

PISTE 2 · CHANGER LE TRAJET DES DONNÉES

Même attention. Moins d’allers-retours.

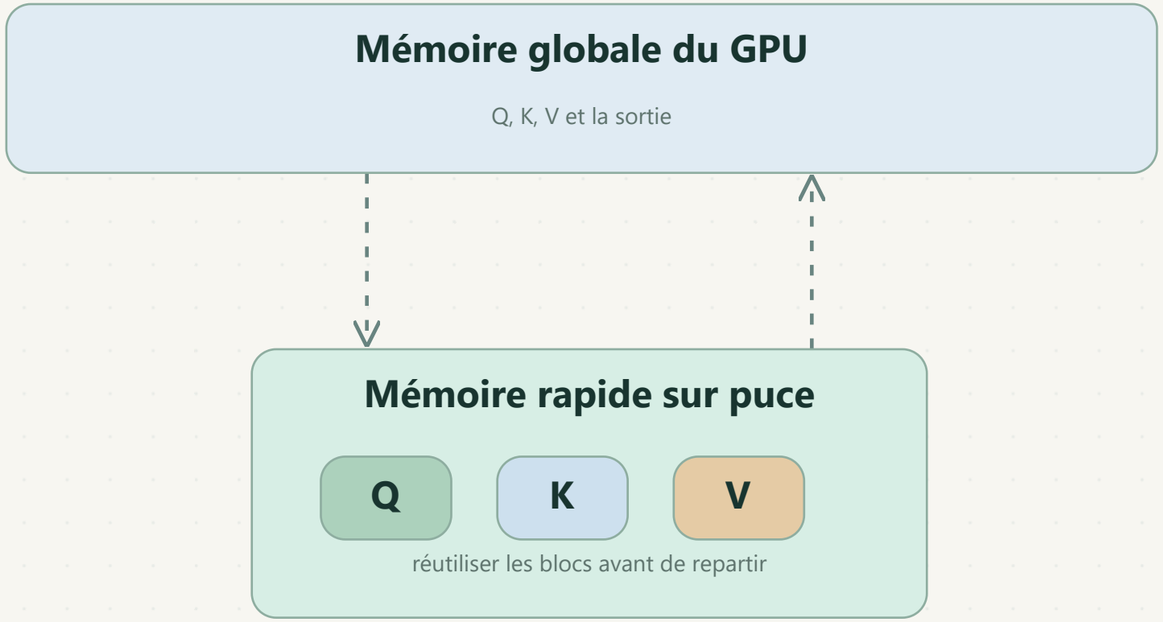
Un GPU est un processeur qui effectue beaucoup de calculs en parallèle. FlashAttention réorganise ses accès mémoire.

> La mémoire rapide est petite : le calcul est organisé en blocs.

> Réutiliser un bloc évite des lectures répétées en mémoire globale.

À RETENIR

L’attention reste complète, à l’arrondi numérique près. FlashAttention ne supprime pas le cache K/V.



Tri Dao et ses coauteurs, 2022. Schéma de principe GPU ; aucun gain automatique n’est promis sur Raspberry Pi.

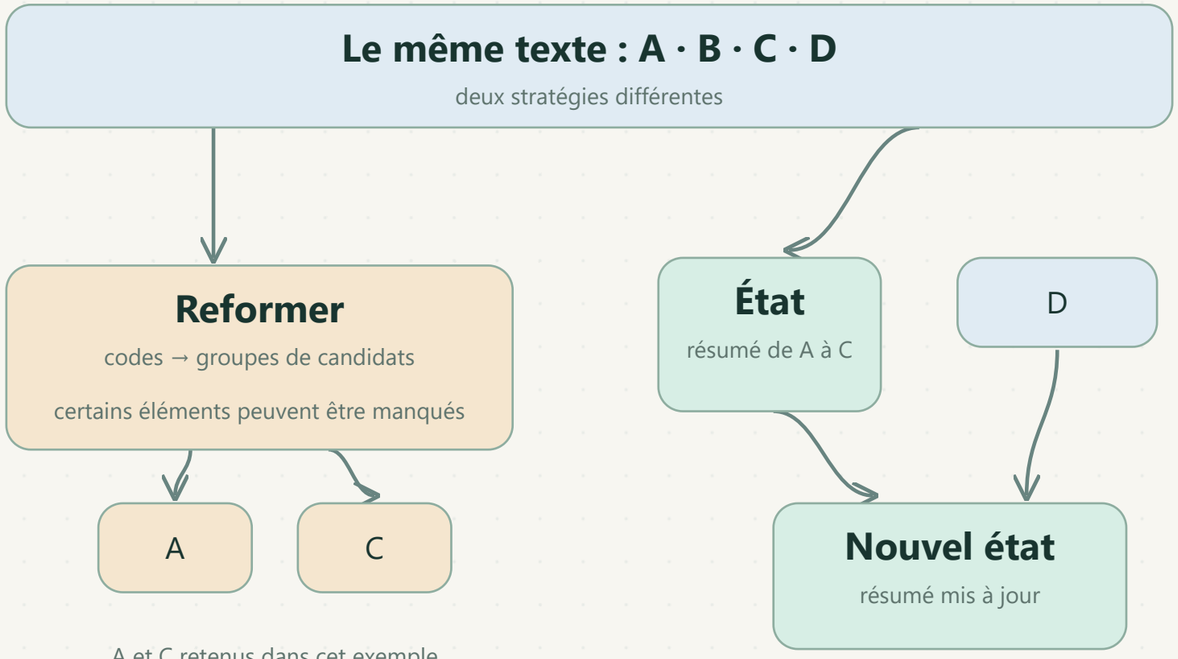
PISTE 3 · CHANGER LA MÉMOIRE DU MODÈLE

Garder des éléments. Ou résumer le passé.

Une autre architecture change la manière de conserver et retrouver le passé.

> **Reformer · 2020 :** le hachage calcule des codes pour regrouper des éléments proches. Certains éléments peuvent être manqués.

> **Mamba · 2023 :** un état est un résumé numérique ; la récurrence le met à jour à chaque token.



Kitaev et al. pour Reformer ; Albert Gu et Tri Dao pour Mamba. Ce ne sont pas des remplacements universellement meilleurs.

À RETENIR

Pour un modèle fixé, l'état de Mamba ne grandit pas avec le contexte. Cela ne préserve pas forcément tous les détails.

LA QUESTION QUI ÉVITE UNE FAUSSE CONCLUSION

L’information utile est dans un autre groupe.

Le hachage de proximité regroupe des vecteurs d’ attention. Si on ne consulte que certains groupes, quel risque reste-t-il ?

A. Toutes les informations seront forcément retrouvées.

B. Une information pertinente peut ne pas être consultée.

C. Des vecteurs proches ne doivent jamais partager un groupe.

Le but est de regrouper des vecteurs proches. Mais un voisin pertinent peut être classé ailleurs et ne pas être consulté.

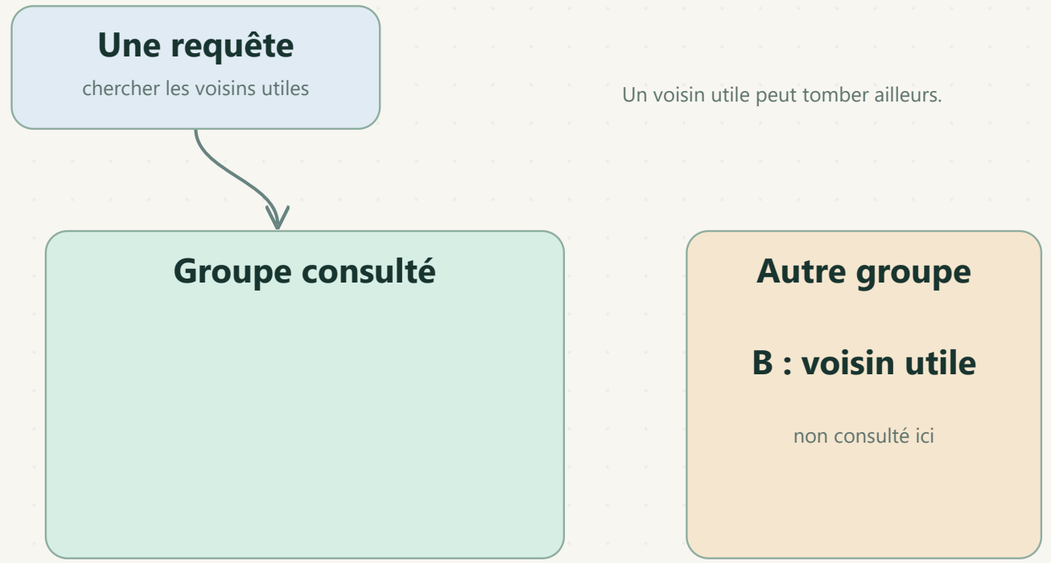


Schéma de sélection approximative de vecteurs, pas un hachage de phrases entières. Reformer utilise des groupes de proximité.

À RETENIR

Ici, l’erreur à tester est une information utile non consultée, pas le simple fait de partager un code.

PISTE 4 · ENTRAÎNER POUR UNE AUTRE ARITHMÉTIQUE

Et avec des poids −1, 0, +1 ?

Ternaire signifie trois valeurs possibles. BitNet b1.58 apprend avec des poids −1, 0 et +1.

$$(-1)\times x_1 + 0\times x_2 + (+1)\times x_3 = -x_1 + x_3$$

x_1, x_2, x_3 : trois valeurs d'entrée.
$\log_2(3)\approx 1{,}58$ bits : limite idéale pour trois choix aussi fréquents.
Échelles et autres données s'ajoutent au stockage réel.

−1
soustraire

0
ignorer

+1
additionner

$$y = -x_1 + x_3$$

Exemple de poids ternaires ; activations non binaires.

À RETENIR

Un noyau est une fonction de calcul adaptée au processeur. Entraînement et noyaux doivent être compatibles.

Apprentissage adapté à la basse précision, pas une quantification post-entraînement arbitraire. Ma et al., 2024 ; modèle 2B4T, 2025.

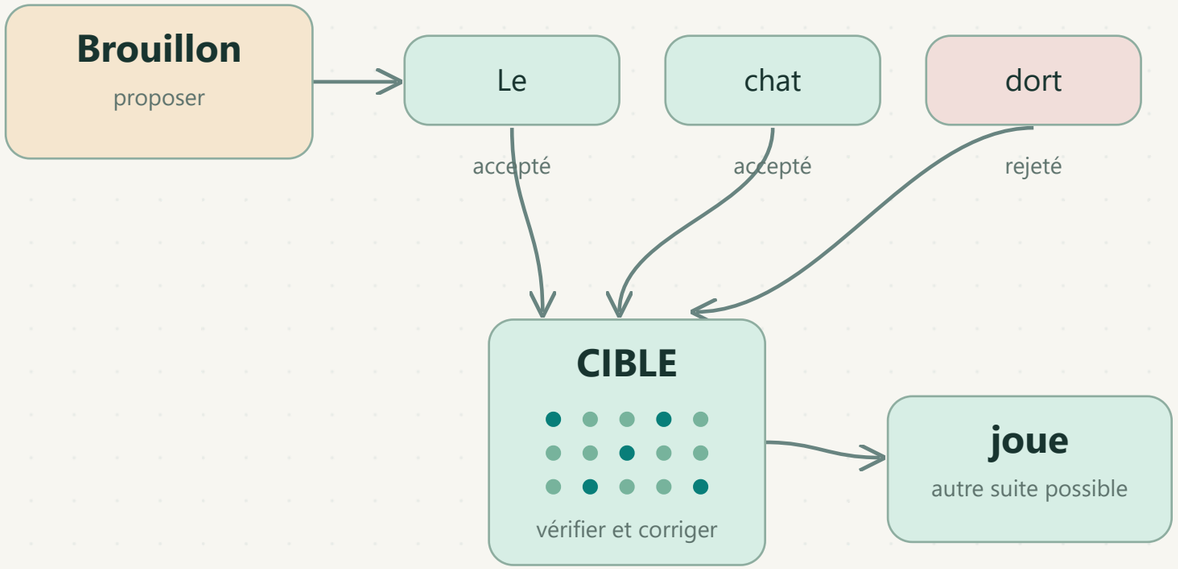
PISTE 5 · PROPOSER, PUIS VÉRIFIER

Un petit modèle écrit. Le grand contrôle.

Le modèle brouillon propose une suite. Le modèle cible, celui que l'on veut reproduire, la vérifie.

> Vérifier plusieurs propositions ensemble peut mieux réutiliser les poids que plusieurs lectures séparées.

> La distribution est la liste des probabilités de sortie. Une correction de tirage conserve ces probabilités dans l'algorithme exact.



La vraie règle porte sur des probabilités, pas sur « cela semble juste ».

À RETENIR

La vérification a un coût. Le gain dépend du brouillon, du matériel et du nombre de tokens acceptés.

Préserver les probabilités ne signifie pas garantir la vérité ni obtenir le même tirage aléatoire. Ce procédé peut aussi ne pas accélérer un petit appareil.

DISTINGUER LES NIVEAUX DE PREUVE

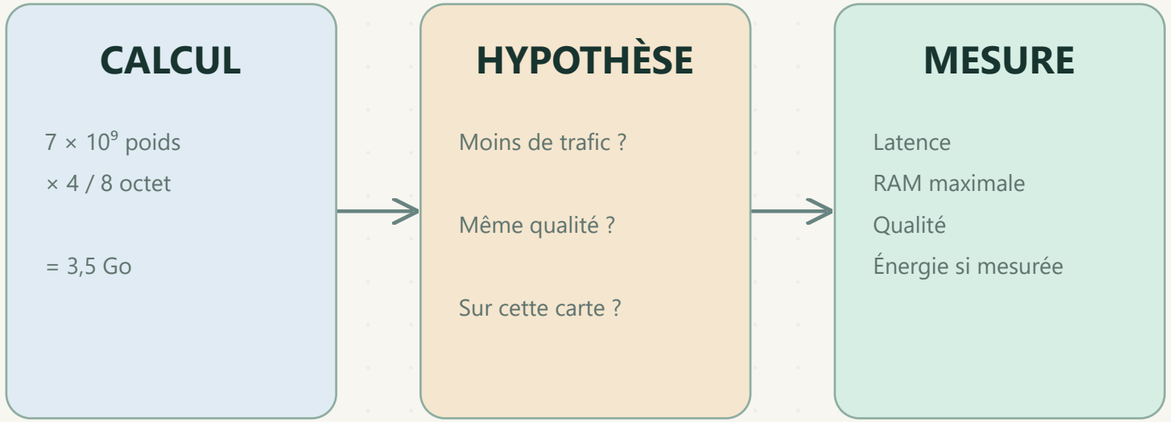
Une équation n’est pas un chronomètre.

Un calcul peut aider à choisir une piste. Il ne démontre pas la qualité d’un modèle ni sa vitesse réelle.

> **Hypothèse :** résultat attendu, écrit avant les mesures.

> **Calculé :** taille des poids ou minimum théorique de temps.

> **Mesuré :** durée, RAM maximale et qualité sur des exemples de test.



Un débit et un score de qualité exigent des expériences.

À RETENIR

Une taille de fichier ne démontre ni un débit réel ni une qualité de réponse.

Qualité : réussite sur la tâche choisie. Le jeu de test reste indépendant des données qui ont servi à choisir les réglages.

EXERCICE DE DÉCISION · DONNÉES FICTIVES

Plus rapide. Mais acceptable ?

Fixons avant l'essai : au moins 47 bonnes réponses sur 50 et une durée médiane d'au plus 150 ms.

> La **médiane** est la valeur du milieu des durées triées. L'intervalle indique ici le minimum et le maximum.

> Même entrée de 128 tokens, réponse de 20 tokens, modèle déjà chargé. Peut-on choisir la variante ?

À RETENIR

Non selon ce cahier des charges : 46/50 est sous le seuil de qualité. Un gain de vitesse ne suffit pas.

EXERCICE · DONNÉES FICTIVES

Indicateur	Référence	Variante
RAM maximale	4,8 Go	3,9 Go
Durée médiane	120 ms 110–145 ms	80 ms 75–105 ms
Réponses correctes	47 / 50	46 / 50

Plus rapide, mais sous le seuil de qualité fixé.

Données fictives. Durée = traitement d'une entrée de 128 tokens + génération de 20 tokens ; chargement exclu ; 5 répétitions. Même test de 50 questions.

DU PC À LA MACHINE VISÉE

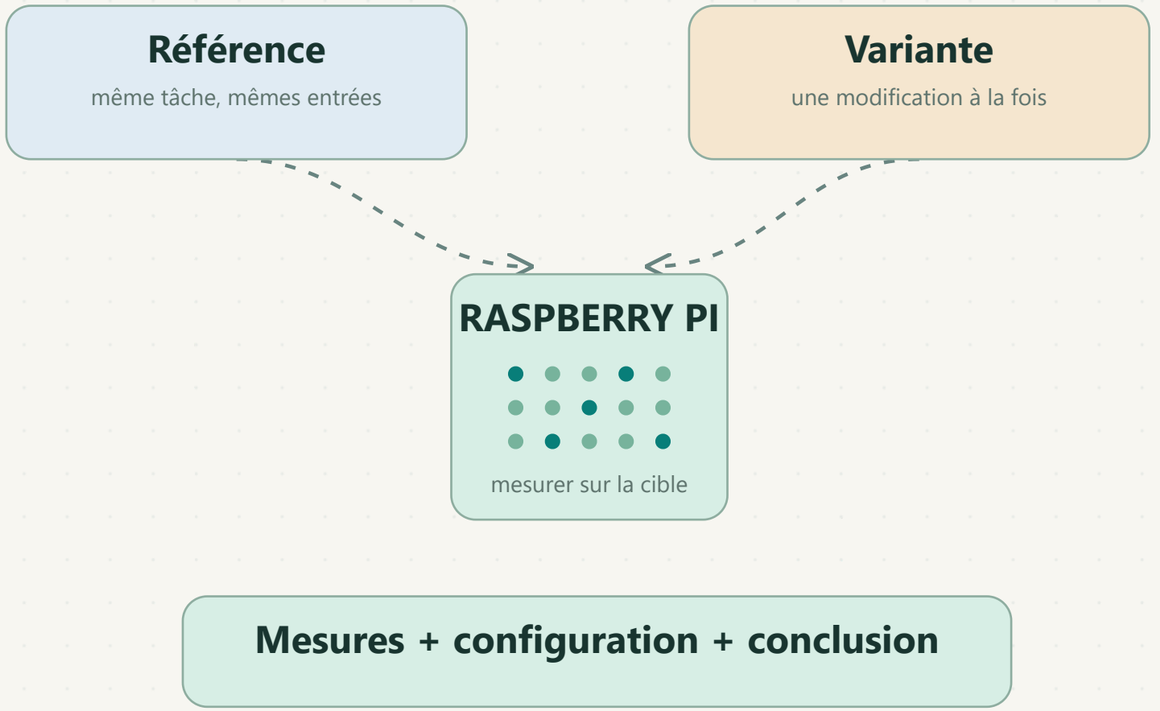
Sur le Pi : choisir, puis vérifier.

Un logiciel d'exécution, ou runtime, charge le modèle et fait ses calculs sur la carte.

> llama.cpp est un logiciel d'exécution ; GGUF est un format de fichier, pas une méthode de quantification.

> Tester aussi contexte, cache pris en charge et nombre de threads, les tâches de calcul simultanées.

> Conserver les mesures brutes en CSV, un tableau en texte. Mesurer premier token, débit, RAM et qualité ; médiane et dispersion ; température contrôlée.



À RETENIR

Référence : version de départ. Variante : une seule modification. Garder les entrées et les critères identiques.

Contrôler température, alimentation, versions et autres programmes. Le protocole technique lié dans les ressources décrit les mesures sur Pi.

TROIS RÉFLEXES D’INGÉNIEUR

Où est l’information ? Quel est le coût ? Quelle est la preuve ?

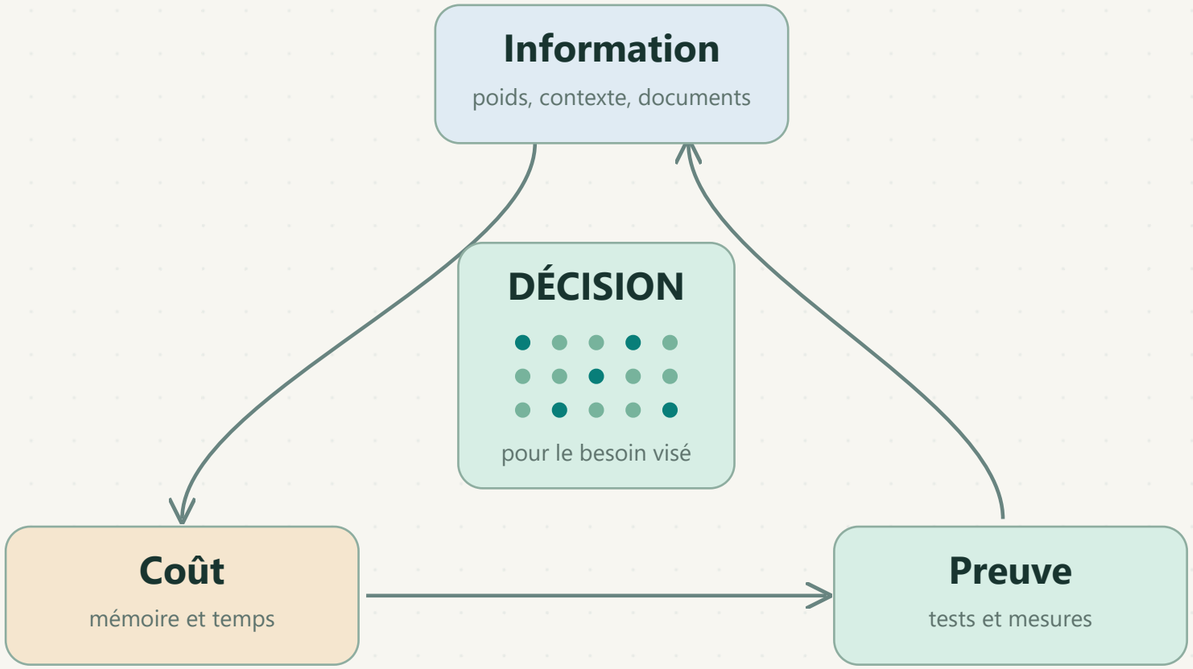
Une optimisation utile respecte le besoin de l’utilisateur et les limites de la machine.

> Les poids portent des régularités apprises ; le contexte et les documents apportent des informations.

> Moins de bits, moins de transferts ou une autre architecture : les compromis doivent être vérifiés.

À RETENIR

Trois questions : quelles informations, quel budget, quelle preuve sur la machine visée ?



Le parcours Ingénieur complète les schémas par les formules, les architectures de recherche et le protocole de mesure.

TEXTES ORIGINAUX

Représenter, apprendre, interpréter.

Des sources pour préciser ce que les schémas simplifient.

R1 · Attention Is All You Need ↗

Ashish Vaswani et al. · 2017
NeurIPS 2017

R2 · Locating and Editing Factual Associations in GPT ↗

Kevin Meng, David Bau, Alex Andonian, Yonatan Belinkov · 2022
NeurIPS 2022

R3 · Toy Models of Superposition ↗

Nelson Elhage et al. · 2022
Article de recherche · modèles jouets

R4 · Extracting Training Data from Large Language Models ↗

Nicholas Carlini et al. · 2021
USENIX Security 2021 · prépublication initiale 2020

R16 · Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks ↗

Patrick Lewis et al. · 2020
NeurIPS 2020

À RETENIR

Les mécanismes observés dans une expérience ne constituent pas une carte complète de tous les modèles.

TEXTES ORIGINAUX

Compter les octets. Changer les calculs.

Un gain publié garde le contexte matériel et le protocole du papier.

R5 · Roofline: An Insightful Visual Performance Model… ↗

Samuel Williams, Andrew Waterman, David Patterson · 2008 / 2009
Rapport Berkeley 2008 · article CACM 2009

R6 · GQA: Training Generalized Multi-Query Transformer Models… ↗

Joshua Ainslie et al. · 2023
EMNLP 2023

R7 · AWQ: Activation-aware Weight Quantization… ↗

Ji Lin et al. · 2023 / 2024
Prépublication 2023 · MLSys 2024

R8 · FlashAttention: Fast and Memory-Efficient Exact Attention… ↗

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré · 2022
NeurIPS 2022

R9 · Reformer: The Efficient Transformer ↗

Nikita Kitaev, Łukasz Kaiser, Anselm Levskaya · 2020
ICLR 2020

R10 · Mamba: Linear-Time Sequence Modeling with Selective State Spaces ↗

Albert Gu, Tri Dao · 2023 / 2024
Prépublication 2023 · COLM 2024

À RETENIR

Les simulations de cette présentation sont des exemples pédagogiques ; aucun LLM n’a été exécuté sur Pi ici.

ARTICLES ET IMPLÉMENTATIONS

Des pistes réelles. Des limites à vérifier.

Sélection de publications et de code ; ce n’est pas un classement exhaustif de la recherche.

R11 · The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits ↗

Shuming Ma et al. · 2024
Prépublication · résultats dans les configurations étudiées

R12 · BitNet b1.58 2B4T Technical Report ↗

Shuming Ma et al. · 2025
Rapport technique · prépublication

R13 · Fast Inference from Transformers via Speculative Decoding ↗

Yaniv Leviathan, Matan Kalman, Yossi Matias · 2023
ICML 2023 · prépublication initiale 2022

R14 · GGUF — spécification du format ↗

Projet ggml · Documentation évolutive
Documentation officielle

R15 · BitNet — implémentation officielle ↗

Microsoft Research · Code évolutif
Dépôt officiel

À RETENIR

Revenir aux données et aux conditions d’essai avant de transporter un résultat vers sa propre machine.